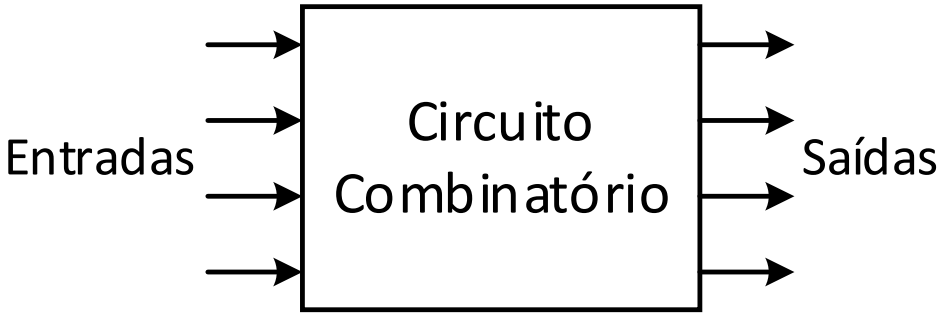
Circuitos Combinatórios

Circuito Combinatório

- Circuito digital pode ser:
  - Combinatório;
  - Sequencial.
- **Circuito combinatório** – os valores das saídas num determinado instante dependem apenas dos valores das entradas nesse instante;
- **Circuito sequencial** – os valores das saídas num determinado instante não dependem apenas dos valores das entradas nesse instante, mas também da sequência de valores ao longo do tempo.

Circuito Combinatório

- **Circuito combinatório** tem um conjunto de entradas e um conjunto de saídas;



- Cada uma das saídas está associada a uma função booleana do conjunto de entradas;
- O circuito é implementado com um conjunto de portas lógicas que implementam as funções.

Síntese de Circuitos Combinatórios Simples

ISEL

- Identificar as entradas e as saídas do circuito;
- Determinar a tabela de verdade ou a expressão lógica de cada uma das saídas do circuito;
- Simplificar as funções de saída;
- Descrever o circuito numa linguagem de descrição de hardware (VHDL);
- Verificar a correção do circuito por simulação;
- Implementar e testar o circuito.

Circuitos Combinatórios Típicos

Circuitos Combinatórios Típicos

ISEL

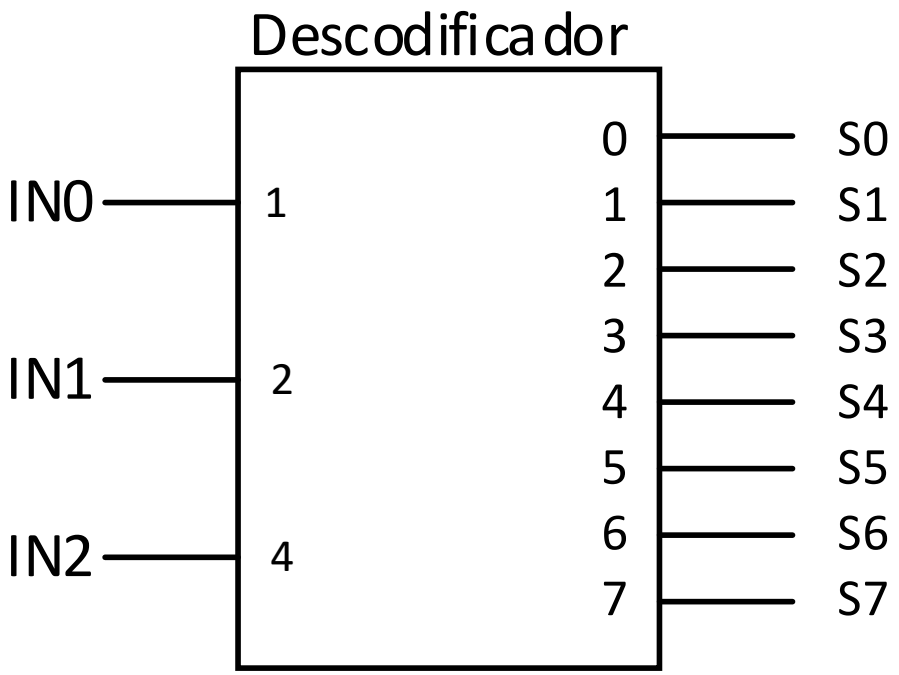
- O projeto modular utiliza vários módulos em hierarquia para descrever o circuito lógico completo;
- Alguns módulos são usados frequentemente:
  - Descodificadores
  - Codificadores
  - Multiplexers
  - Demultiplexers

Descodificador

- Um descodificador permite identificar uma palavra binária de um determinado código;
- O descodificador recebe a palavra binária e tem tantas saídas quanto o número de códigos;
- Um descodificador binário de *n* bits tem *n* entradas para receber as palavras de *n* bits e 2^n saídas, em que apenas ativa a saída correspondente ao código de entrada.

Descodificador Binário de 3 bits

- Consideremos um descodificador binário de 3 bits



IN2 IN1 IN0	S0	S1	S2	S3	S4	S5	S6	S7
0 0 0	1	0	0	0	0	0	0	0
0 0 1	0	1	0	0	0	0	0	0
0 1 0	0	0	1	0	0	0	0	0
0 1 1	0	0	0	1	0	0	0	0
1 0 0	0	0	0	0	1	0	0	0
1 0 1	0	0	0	0	0	1	0	0
1 1 0	0	0	0	0	0	0	1	0
1 1 1	0	0	0	0	0	0	0	1

Descodificador Binário – Funções Lógicas

ISEL

IN2 IN1 IN0	S0	S1	S2	S3	S4	S5	S6	S7
0 0 0	1	0	0	0	0	0	0	0
0 0 1	0	1	0	0	0	0	0	0
0 1 0	0	0	1	0	0	0	0	0
0 1 1	0	0	0	1	0	0	0	0
1 0 0	0	0	0	0	1	0	0	0
1 0 1	0	0	0	0	0	1	0	0
1 1 0	0	0	0	0	0	0	1	0
1 1 1	0	0	0	0	0	0	0	1

$S0 = \overline{IN2}\ \overline{IN1}\ \overline{IN0}$

$S1 = \overline{IN2}\ \overline{IN1}\ IN0$

$S2 = \overline{IN2}\ IN1\ \overline{IN0}$

$S3 = \overline{IN2}\ IN1\ IN0$

$S4 = IN2\ \overline{IN1}\ \overline{IN0}$

$S5 = IN2\ \overline{IN1}\ IN0$

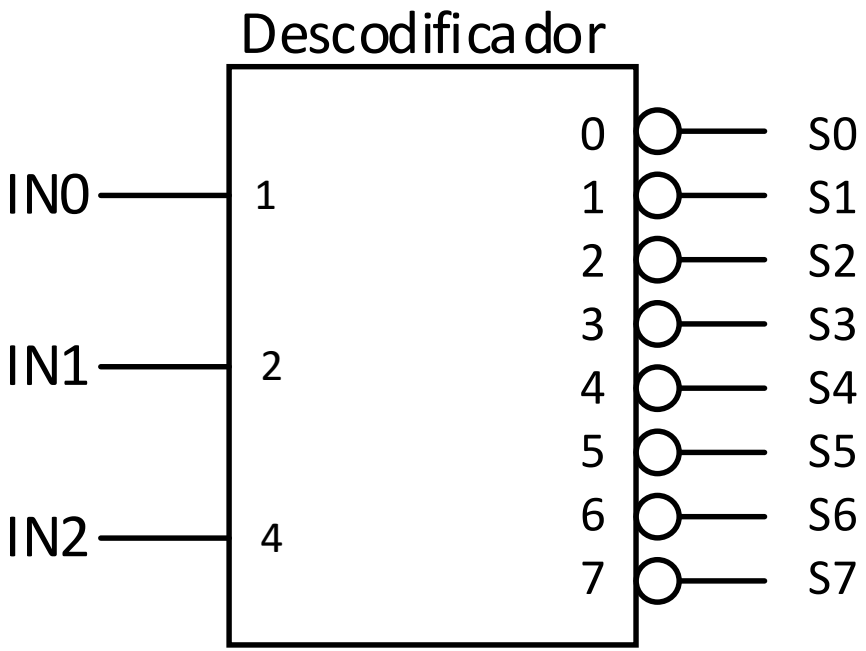
$S6 = IN2\ IN1\ \overline{IN0}$

$S7 = IN2\ IN1\ IN0$

Saídas Ativas a LOW e a HIGH

- As saídas e entradas de um circuito podem ser ativas a HIGH ou a LOW:
  - Ativas a HIGH, ativas com valor lógico 1;
  - Ativas a LOW, ativas com valor lógico 0;
- As entradas ou saídas ativas a LOW são indicadas com um círculo;
- As entradas ou saídas ativas a HIGH são indicadas sem círculo;

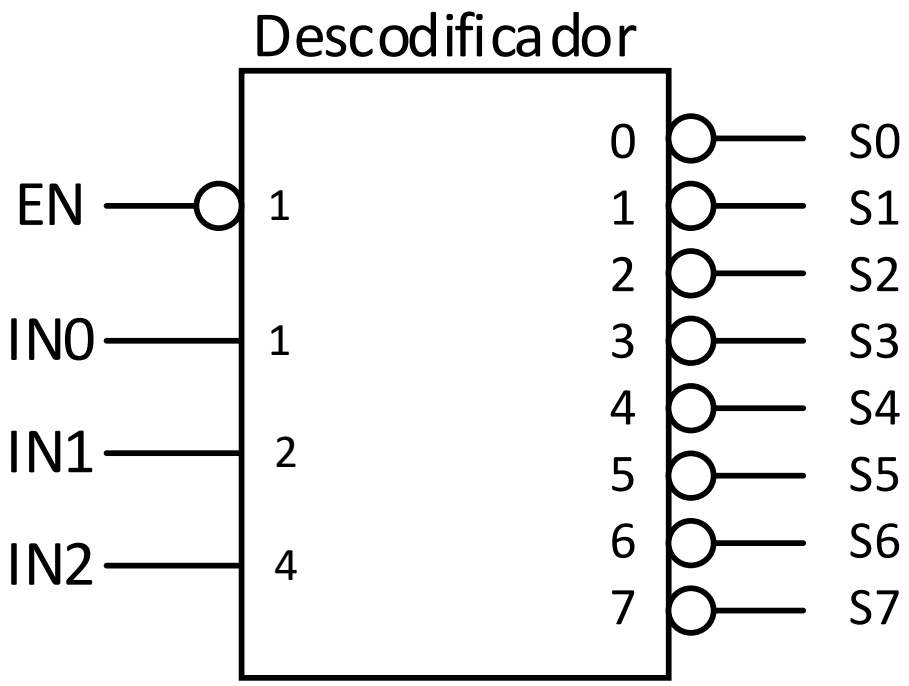
Descodificadores com Saídas Ativas a LOW



IN2 IN1 IN0	S0	S1	S2	S3	S4	S5	S6	S7
0 0 0	0	1	1	1	1	1	1	1
0 0 1	1	0	1	1	1	1	1	1
0 1 0	1	1	0	1	1	1	1	1
0 1 1	1	1	1	0	1	1	1	1
1 0 0	1	1	1	1	0	1	1	1
1 0 1	1	1	1	1	1	0	1	1
1 1 0	1	1	1	1	1	1	0	1
1 1 1	1	1	1	1	1	1	1	0

Descodificador com *Enable*

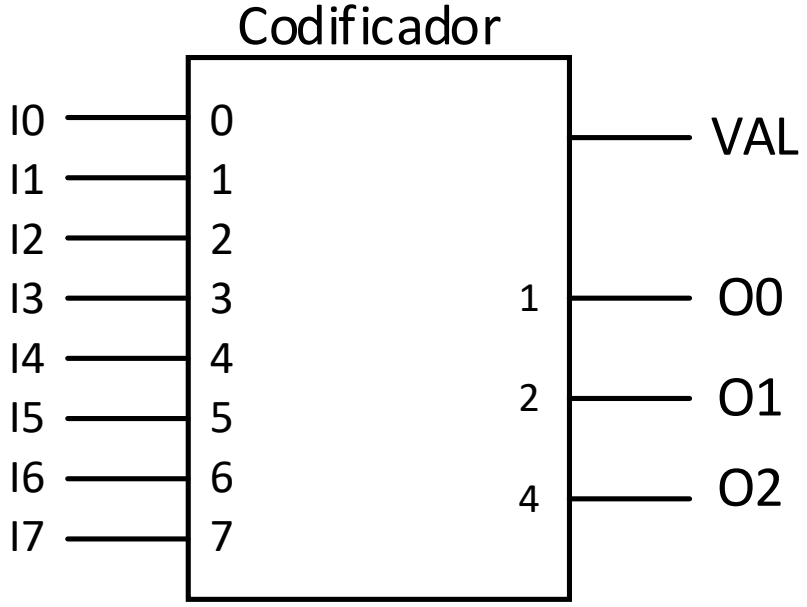
A entrada de *Enable* permite desativar o descodificador



EN IN2 IN1 IN0	S0	S1	S2	S3	S4	S5	S6	S7
1 - - -	1	1	1	1	1	1	1	1
0 0 0 0	0	1	1	1	1	1	1	1
0 0 0 1	1	0	1	1	1	1	1	1
0 0 1 0	1	1	0	1	1	1	1	1
0 0 1 1	1	1	1	0	1	1	1	1
0 1 0 0	1	1	1	1	0	1	1	1
0 1 0 1	1	1	1	1	1	0	1	1
0 1 1 0	1	1	1	1	1	1	0	1
0 1 1 1	1	1	1	1	1	1	1	0

Codificador Binário

- Tem uma função inversa ao descodificador;
- Tem tantas entradas quanto o número de códigos e tantas saídas quanto o número de bits da palavra de código;



S0	S1	S2	S3	S4	S5	S6	S7	O2	O1	O0	VAL
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

Codificador de Prioridades

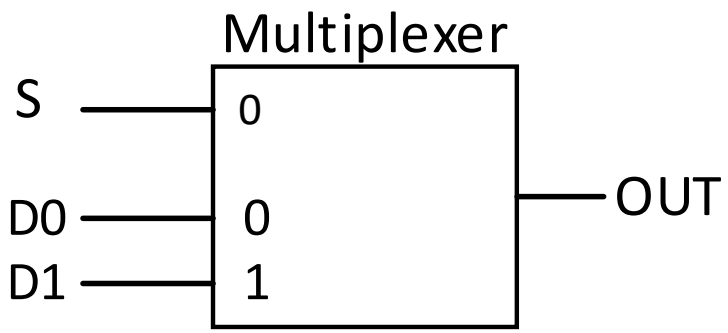
Aplica-se quando há mais de uma entrada ativa.

Ativa a saída correspondente à entrada mais prioritária.

S0	S1	S2	S3	S4	S5	S6	S7	O2	O1	O0	VAL
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
-	1	0	0	0	0	0	0	0	0	1	1
-	-	1	0	0	0	0	0	0	1	0	1
-	-	-	1	0	0	0	0	0	1	1	1
-	-	-	-	1	0	0	0	1	0	0	1
-	-	-	-	-	1	0	0	1	0	1	1
-	-	-	-	-	-	1	0	1	1	0	1
-	-	-	-	-	-	-	1	1	1	1	1

Multiplexer

- Um multiplexer é um circuito que seleciona uma das entradas;
- Um multiplexer tem 2^n entradas de dados, n entradas de seleção e uma saída;
- Por exemplo, um multiplexer de 2 entradas de dados:

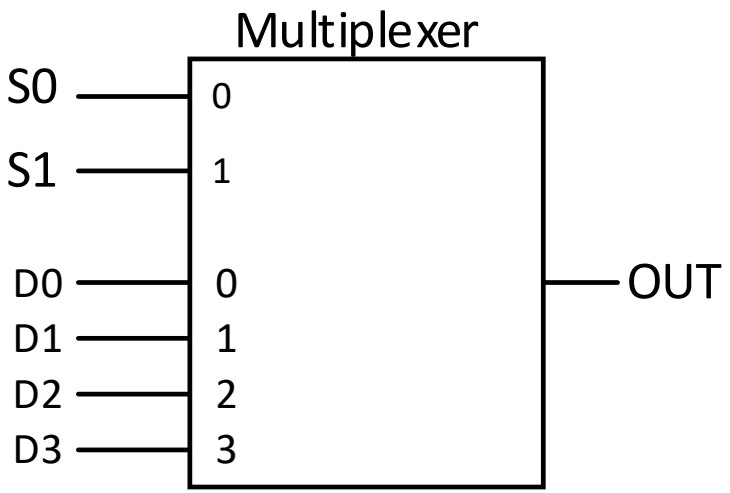


S	OUT
0	D0
1	D1

$$\text{OUT} = \bar{S}\, D0 + S\, D1$$

Multiplexer de 4 Entradas

- Um multiplexer de 4 entradas de dados tem 2 entradas de seleção:



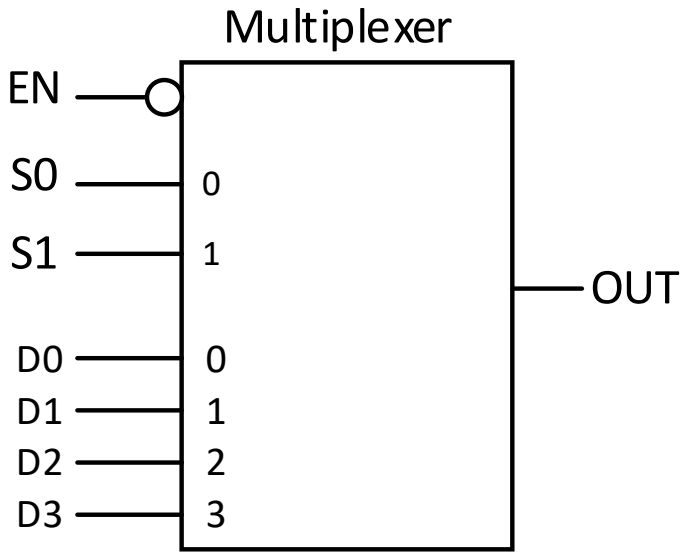
S1	S0	OUT
0	0	D0
0	1	D1
1	0	D2
1	1	D3

$$\mathrm{OUT} = \overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,D0 + \overline{\mathrm{S1}}\,S0\,D1 + S1\,\overline{\mathrm{S0}}\,D2 + \mathrm{S1}\,\mathrm{S0}\,D3$$

Multiplexer com *Enable*

- Um multiplexer também pode ser controlado por um *enable*;
- Um enable inativo coloca a saída inativa;
- Por exemplo, um multiplexer de 4 entradas de dados com enable:

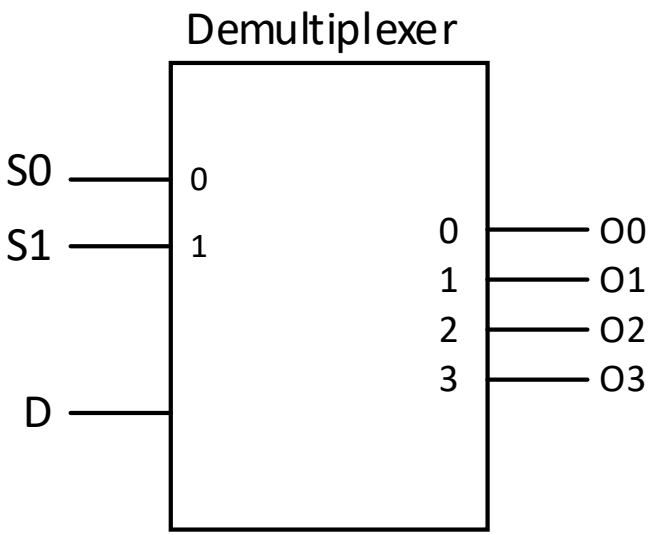
$$\text{OUT} = \overline{EN}\ (\overline{\text{S1}}\ \overline{\text{S0}}\ D0 + \overline{\text{S1}}\ S0\ D1 + S1\ \overline{\text{S0}}\ D2 + \text{S1 S0}\ D3)$$



EN	S1	S0	OUT
1	-	-	0
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3

Demultiplexer

- Um demultiplexer tem uma funcionalidade contrária à do multiplexer;
- Um demultiplexer tem uma entrada, *n* entradas de seleção e 2^n saídas;
- Um demultiplexer encaminha o valor de entrada para a saída selecionada;
- Um demultiplexer é igual a um descodificador com *enable*, em que a entrada de *enable* é usada como entrada de dados



S1	S0	O3	O2	O1	O0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

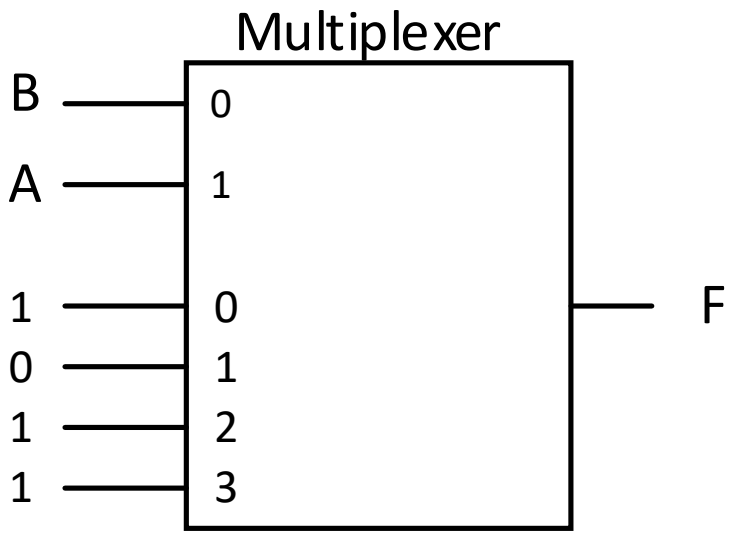
Implementação de Funções com Multiplexer

- Um multiplexer também pode ser utilizado para implementar funções lógicas;
- Um multiplexer gera internamente uma soma de produtos das variáveis de seleção:

$$F = \overline{S1}\,\overline{S0}\,D0 + \overline{S1}\,S0\,D1 + S1\,\overline{S0}\,D2 + S1\,S0\,D3$$

Por exemplo:

A	B	F
0	0	1
0	1	0
1	0	1
1	1	1



Projeto de Circuitos Combinatórios

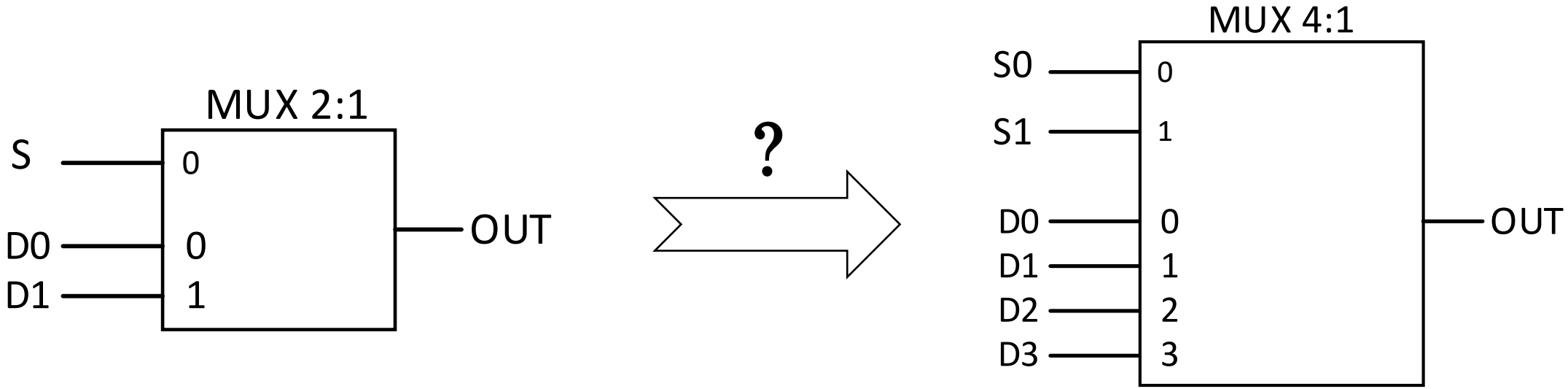
Projeto de Circuitos Combinatórios

ISEL

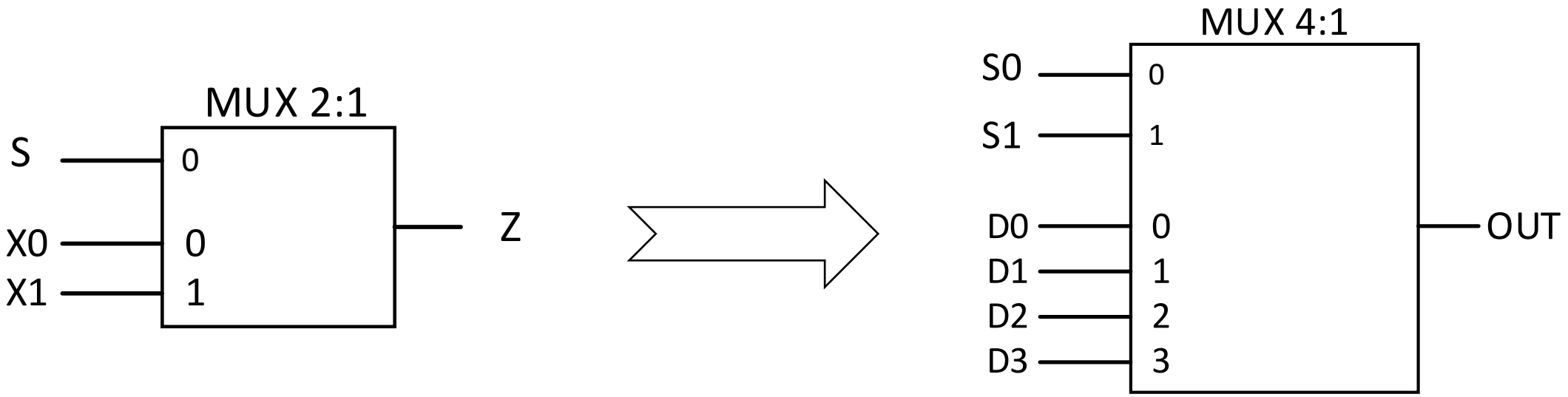
- O projeto de um circuito combinatório com um número reduzido de entradas e de saídas pode ser visto como um todo;
- Para circuitos mais complexos, começamos por dividir o problema em módulos mais simples que depois são interligados para chegar ao circuito completo;
- A subdivisão do circuito pode ter varios níveis de hierarquia. Os módulos do nível final podem ser projetados com as ferramentas que aprendemos.

Projeto Hierárquico – Exemplo Multiplexer

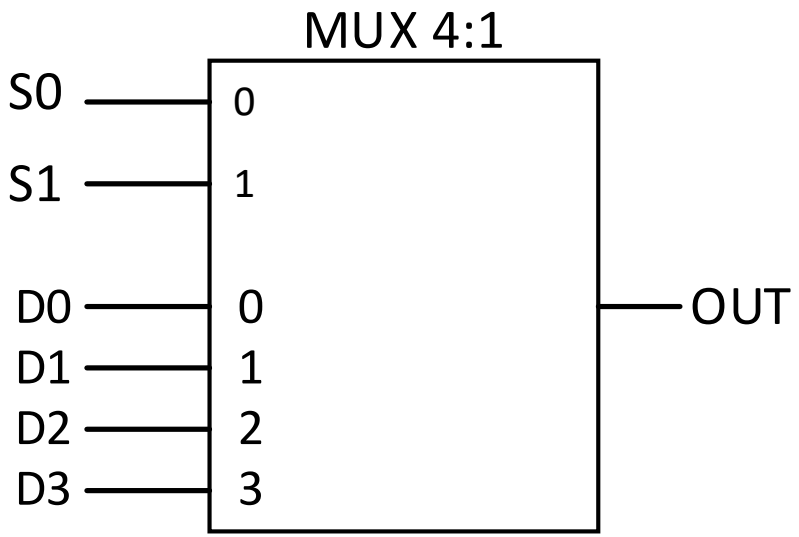
- Considere multiplexers de 2 entradas de dados
- Implemente um multiplexer de 4 entradas de dados

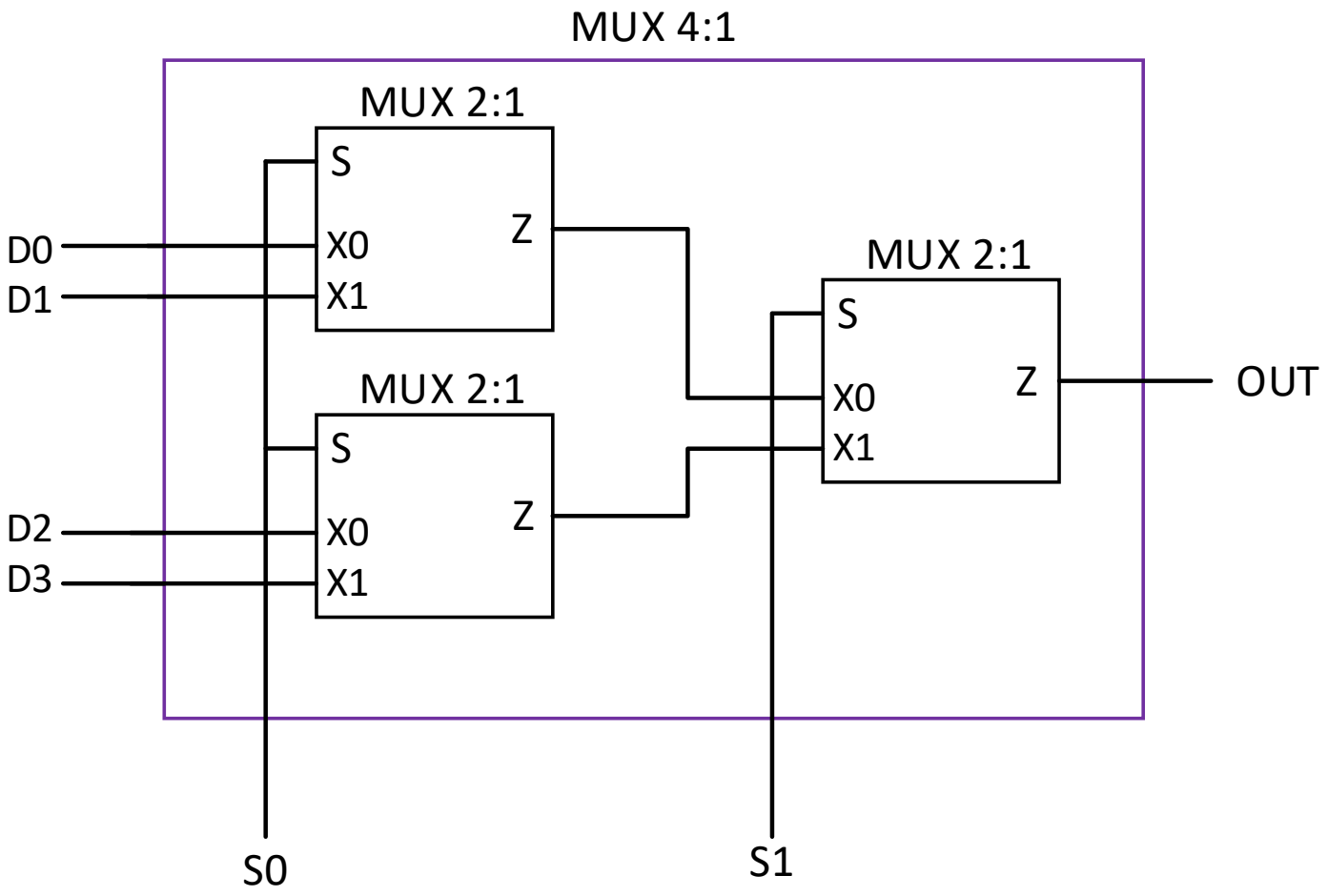


Projeto Hierárquico – Exemplo Multiplexer



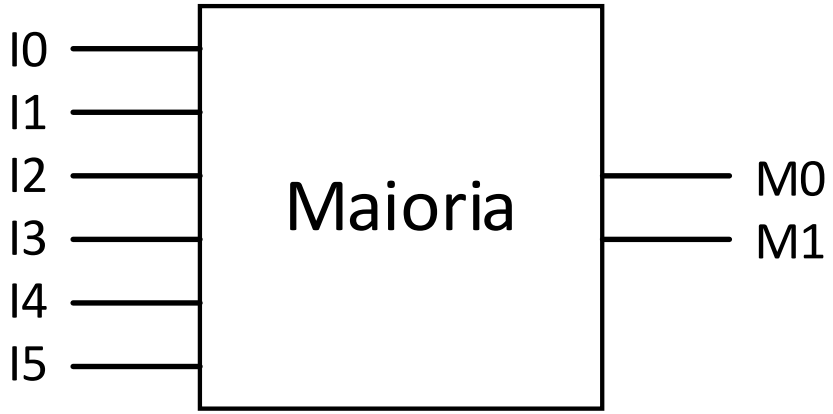
Projeto Hierárquico – MUX 4:1





Projeto Hierárquico – Exemplo Maioria

- Implemente um circuito que indica se existem mais ‘1’s ou ‘0’s numa palavra de 6 bits. A saída de 2 bits será:
- 01 se ‘1’s > ‘0’s
- 11 se ‘1’s < ‘0’s
- 00 se ‘1’s = ‘0’s
- Por exemplo, para uma entrada “101110010”, a saída é “01”

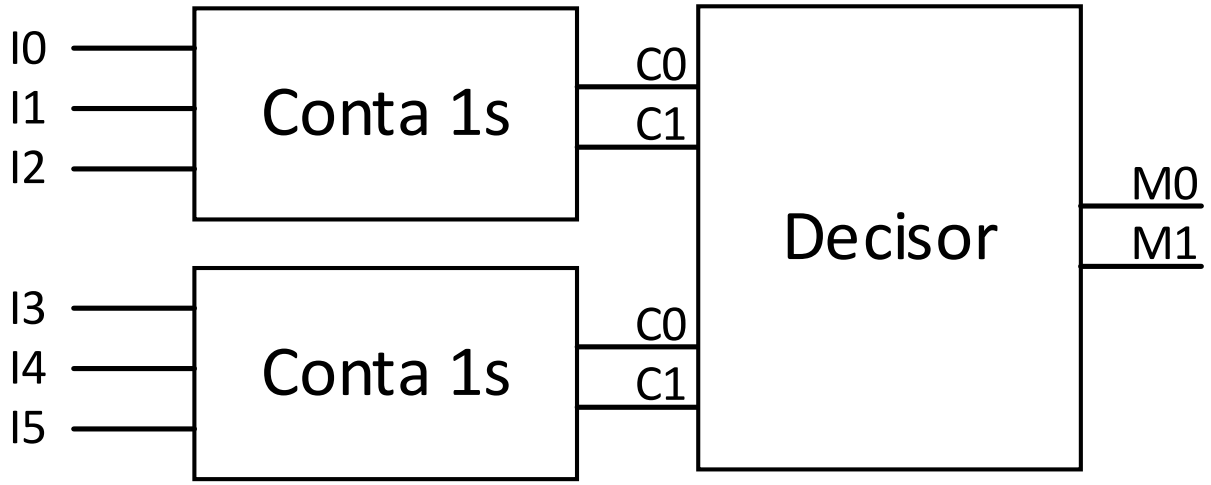


- Tabela de verdade com 6 entradas.

  Demasiado trabalhoso!

Reduzir Complexidade - Dividir o Problema

- Vamos reduzir a complexidade considerando blocos mais pequenos;



- Duas funções de 3 entradas e uma de 4 entradas:

Conta 1s

A B C	Y X
0 0 0	0 0
0 0 1	0 1
0 1 0	0 1
0 1 1	1 0
1 0 0	0 1
1 0 1	1 0
1 1 0	1 0
1 1 1	1 1

$$X = A \oplus B \oplus C$$
$$Y = AB + AC + BC$$

Decisor

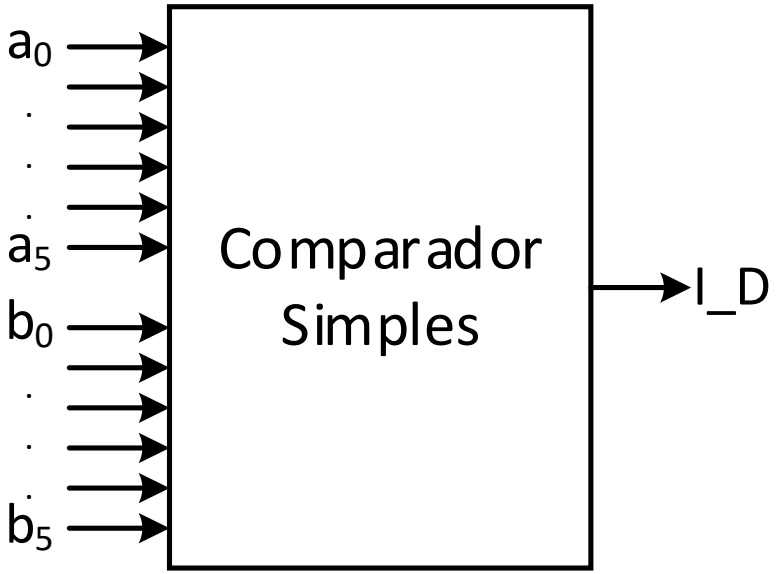
A	B	C	D	M1	M0
0	0	0	0	1	1
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	1	0	1
1	0	0	0	1	1
1	0	0	1	0	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	1

$$P0 = \bar{A}\bar{C} + BD + AC + \bar{B}\bar{D}$$

$$P1 = \bar{A}\bar{C} + \bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{D}$$

Comparador Simples

- Projetar circuito que verifica se duas palavras são diferentes ou iguais. As palavras têm 6 bits;
- A saída fica a:
  - 1 se forem iguais;
  - 0 se forem diferentes.



Comparador Simples - Solução

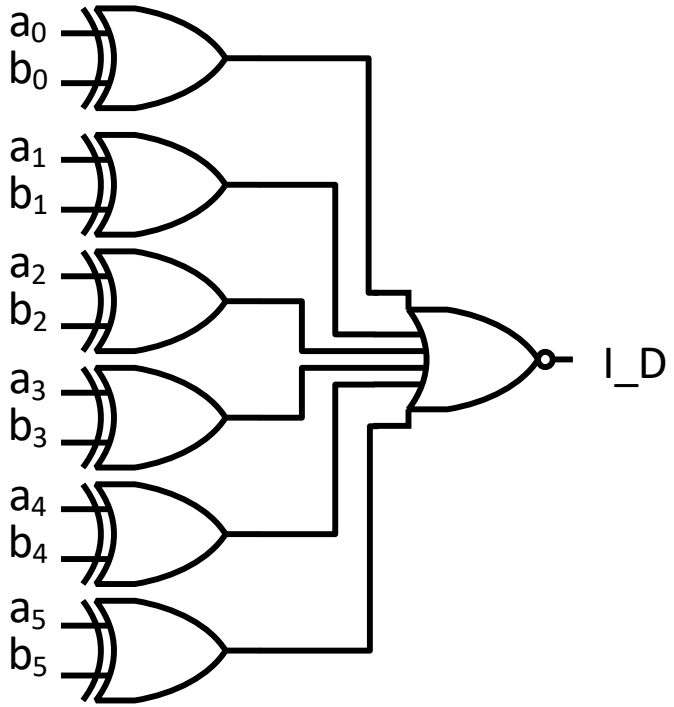
- Vamos comparar bit a bit e depois juntamos os resultados individuais;
- Para comparer dois bits usamos o XOR:

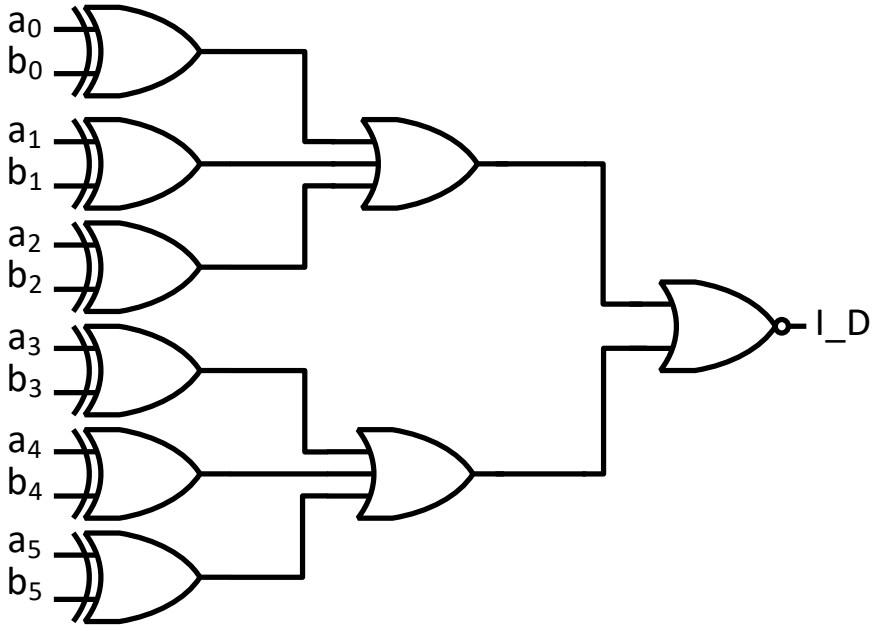
  $A \oplus B = 1$ -- Diferentes

  $A \oplus B = 0$ -- Iguais
- As duas palavras são iguais apenas se todos os pares de bits forem iguais (função NOR).

Comparador Simples – Circuito Lógico

NOR de 6 entradas!





Descrição de Circuitos Combinatórios em VHDL

Prof. Mário Véstias

Descrição Estrutural

- A descrição de um circuito combinatório em VHDL pode compreender apenas uma função lógica (circuitos combinatórios simples).
- Um circuito combinatório é geralmente constituído por vários circuitos mais simples interligados entre si.
- A linguagem VHDL permite descrever o circuito com várias saídas;
- A linguagem VHDL permite descrever o circuito como uma interligação de vários componentes – descrição estrutural;
- Descreve-se cada um dos componentes em funções lógicas e depois descreve-se como os blocos estão interligados;

Declaração de Componentes

- Uma determinada entidade pode ser usada como um **componente** dentro de outra entidade;
- O componente deve ser **declarado** e depois **instanciado** uma ou mais vezes;
- Uma determinada entidade pode ser definida como a interligação de vários componentes;
- Uma determinada entidade pode ser definida como a interligação de componentes e atribuições lógicas;

Declarar Componente em VHDL

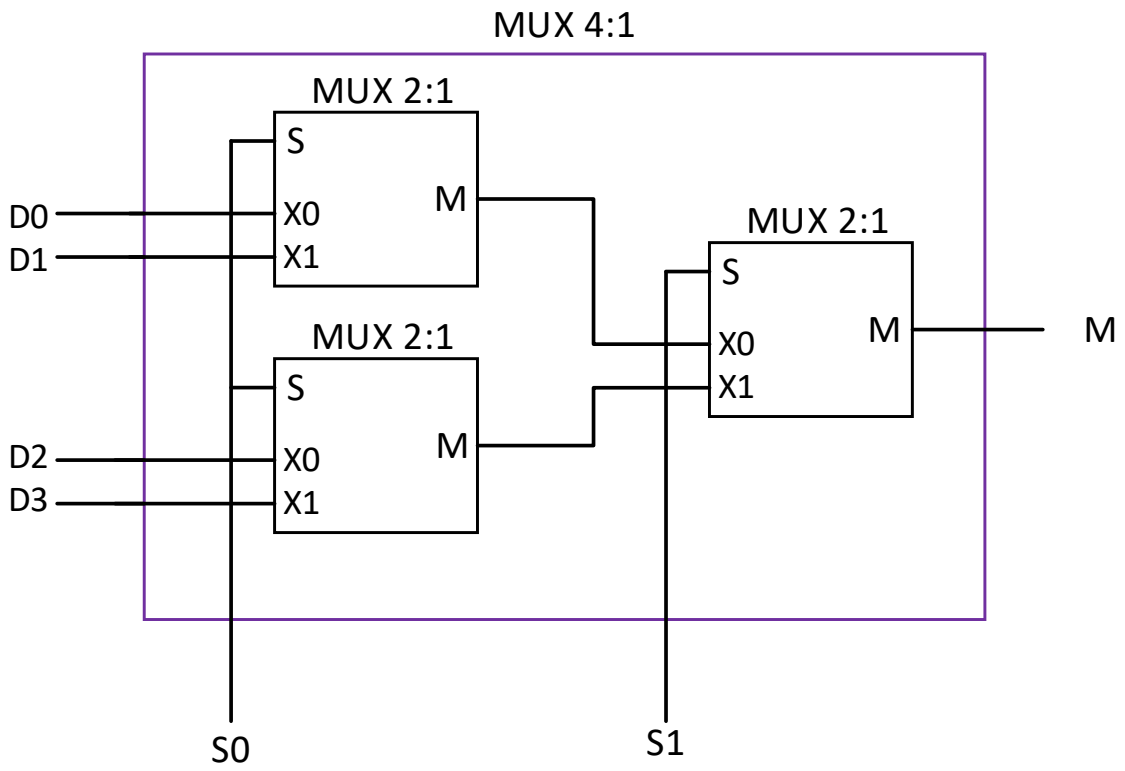
```
entity circuitoTopo is
  port (...
  );
end circuitoTopo;

architecture nome_arquitetura of circuitoTopo is

COMPONENT NOME_DO_COMPONENTE
port (...
  );
END COMPONENT;

-- declaração de sinais

begin
-- descrição do circuito digital
end nome_arquitetura;
```

Instanciar Componente em VHDL

```
architecture nome_arquitetura of circuitoTopo is

COMPONENT NOME_DO_COMPONENTE
port (...
  );
END COMPONENT;

-- declaração de sinais

begin

NOME_INSTANCIA: NOME_DO_COMPONENTE port map(
  PORTO_COMPONENTE => SINAL_CIRCUITO;
  PORTO_COMPONENTE => SINAL_CIRCUITO;
  ...
);
end nome_arquitetura;
```

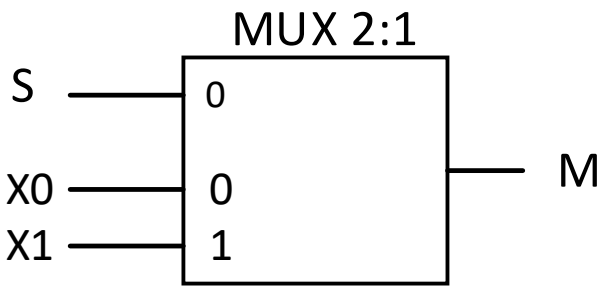
Exemplo de Descrição Estrutural em VHDL

Circuito Lógico – MUX4:1 com MUX2:1



Descrição do Circuito MUX2:1

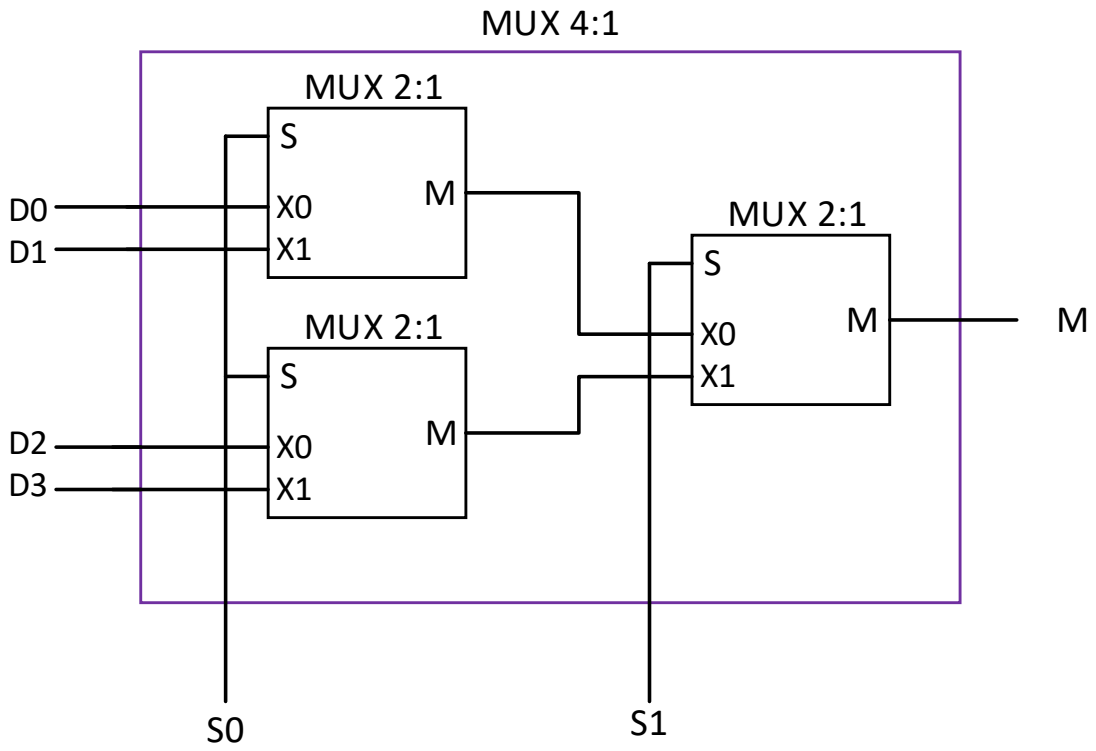
```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity MUX21 is
  port(X : in STD_LOGIC_VECTOR(1 downto 0);
       S : in STD_LOGIC;
       M : out STD_LOGIC
  );
end MUX21;

architecture arq_mux21 of MUX21 is

begin

M <= (not S and X(0)) or (S and X(1));

end arq_mux21;
```



$$M = \bar{S}\,X0 + S\,X1$$

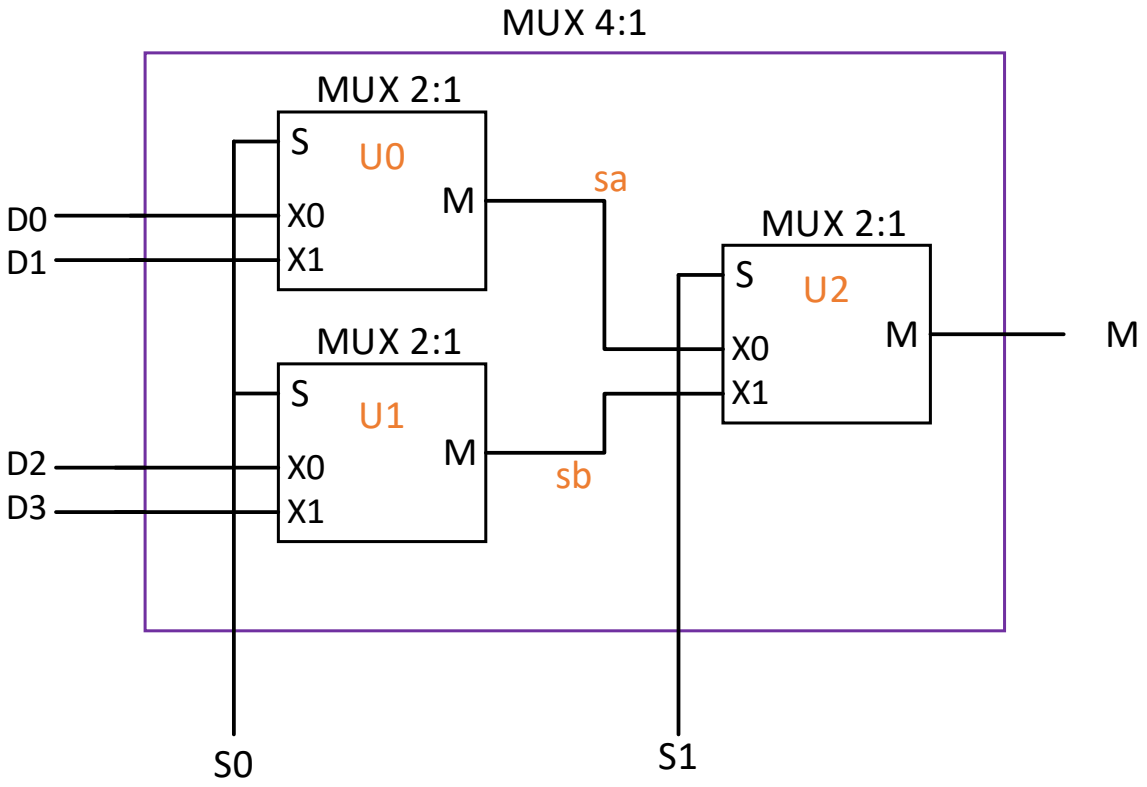
Descrição do Circuito MUX4:1

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity MUX41 is
  port(D : in STD_LOGIC_VECTOR(3 downto 0);
       S : in STD_LOGIC_VECTOR(1 downto 0);
       M : out STD_LOGIC
  );
end MUX41;

architecture arq_mux41 of MUX41 is

component MUX21
  port(X : in STD_LOGIC_VECTOR(1 downto 0);
       S : in STD_LOGIC;
       M : out STD_LOGIC);
end component;
```

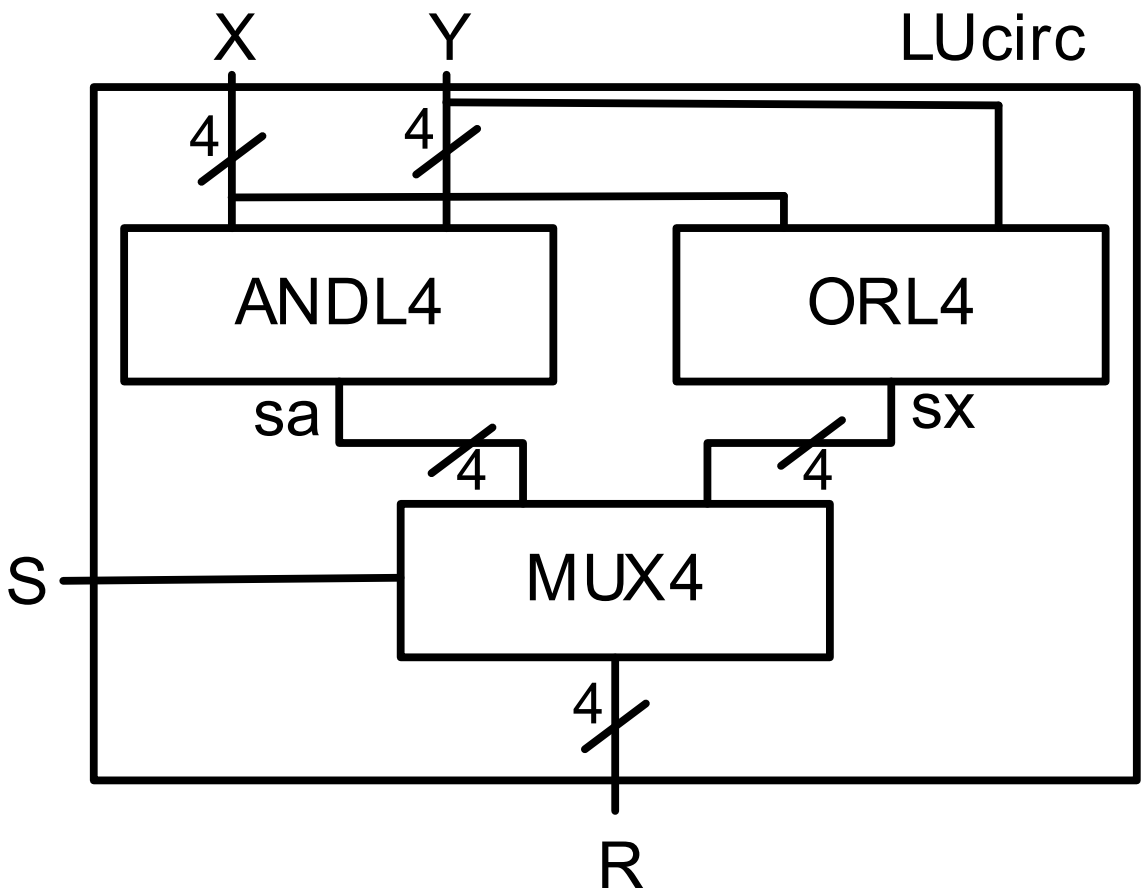


Descrição do Circuito MUX4:1

```
SIGNAL sa, sb : std_logic;

BEGIN

U0: MUX21 port map (
        X(0) => D(0),
        X(1) => D(1),
        S => S(0),
        M => sa);

U1: MUX21 port map (
        X(0) => D(2),
        X(1) => D(3),
        S => S(0),
        M => sb);

U2: MUX21 port map (
        X(0) => sa,
        X(1) => sb,
        S => S(1),
        M => M);

end arq_mux41;
```



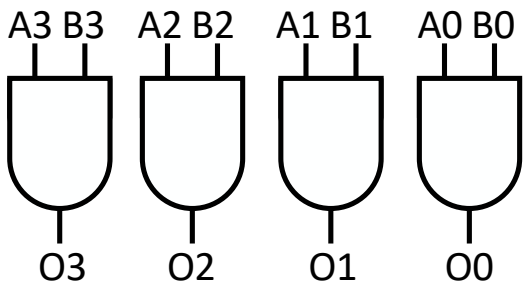
Circuito Lógico - LUcirc



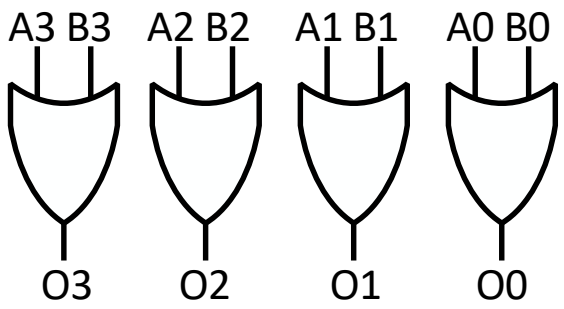
Circuito Lógico - LUcirc

ISEL

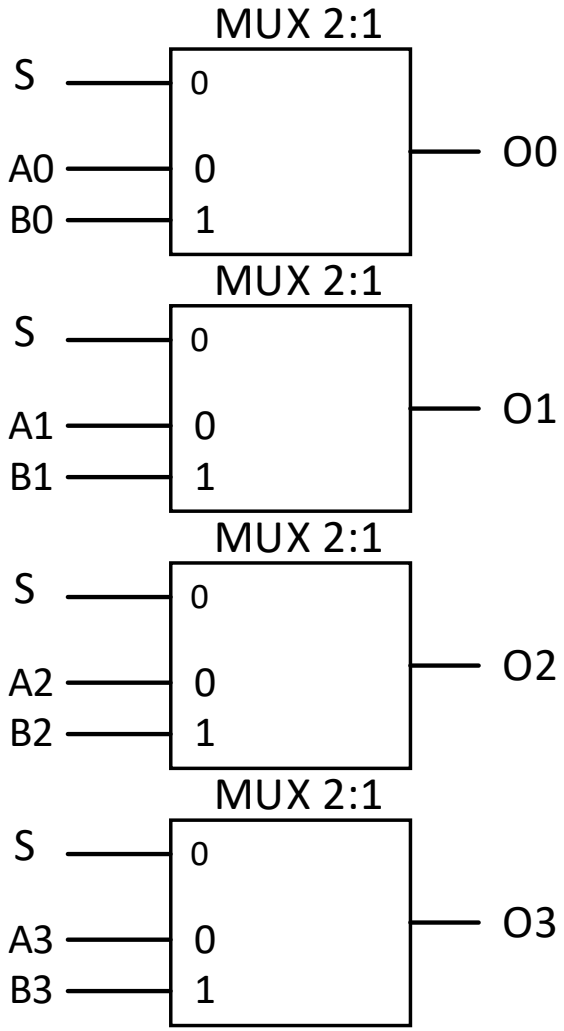
ANDL4



ORL4

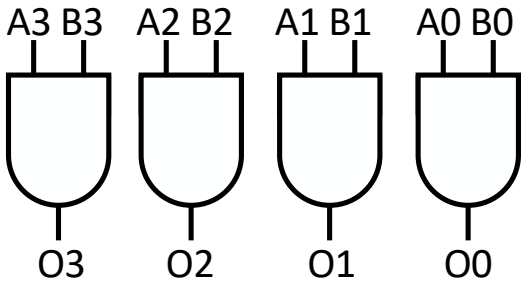


MUX4



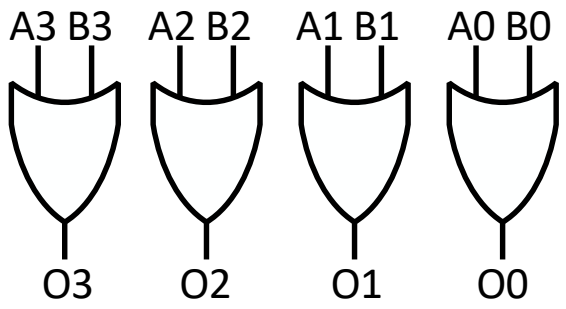
Descrição do Circuito ANDL4

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity ANDL4 is
  port(A : in STD_LOGIC_VECTOR(3 downto 0);
       B : in STD_LOGIC_VECTOR(3 downto 0);
       O : out STD_LOGIC_VECTOR(3 downto 0)
  );
end ANDL4;

architecture arq_andl4 of ANDL4 is

begin
O(0) <= A(0) and B(0);
O(1) <= A(1) and B(1);
O(2) <= A(2) and B(2);
O(3) <= A(3) and B(3);
end arq_andl4;
```



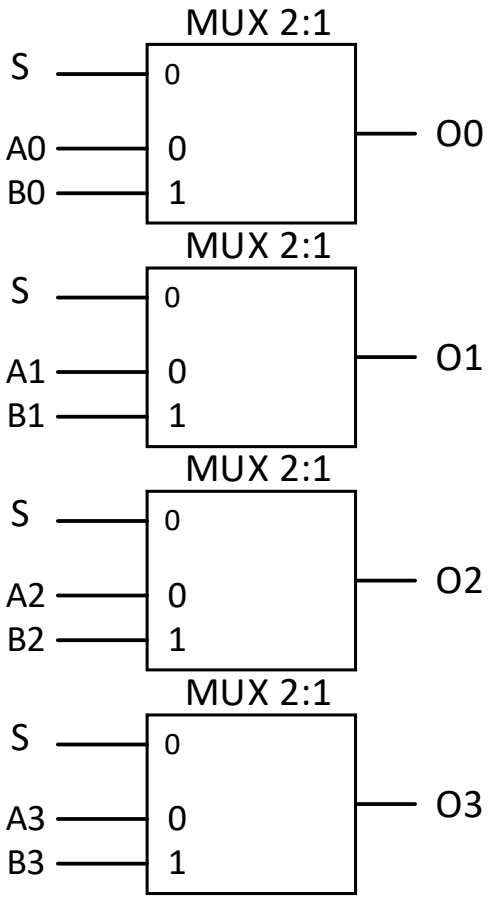
Descrição do Circuito ORL4

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity ORL4 is
  port(A : in STD_LOGIC_VECTOR(3 downto 0);
       B : in STD_LOGIC_VECTOR(3 downto 0);
       O : out STD_LOGIC_VECTOR(3 downto 0)
  );
end ORL4;

architecture arq_orl4 of ORL4 is

begin
O(0) <= A(0) or B(0);
O(1) <= A(1) or B(1);
O(2) <= A(2) or B(2);
O(3) <= A(3) or B(3);
end arq_orl4;
```



Descrição do Circuito MUX4

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity MUX4 is
  port(A : in STD_LOGIC_VECTOR(3 downto 0);
       B : in STD_LOGIC_VECTOR(3 downto 0);
       S : in std_logic;
       O : out STD_LOGIC_VECTOR(3 downto 0)
  );
end MUX4;

architecture arq_mux4 of mux4 is

begin
O(0) <= (not S and A(0)) or (S and B(0));
O(1) <= (not S and A(1)) or (S and B(1));
O(2) <= (not S and A(2)) or (S and B(2));
O(3) <= (not S and A(3)) or (S and B(3));
end arq_mux4;
```



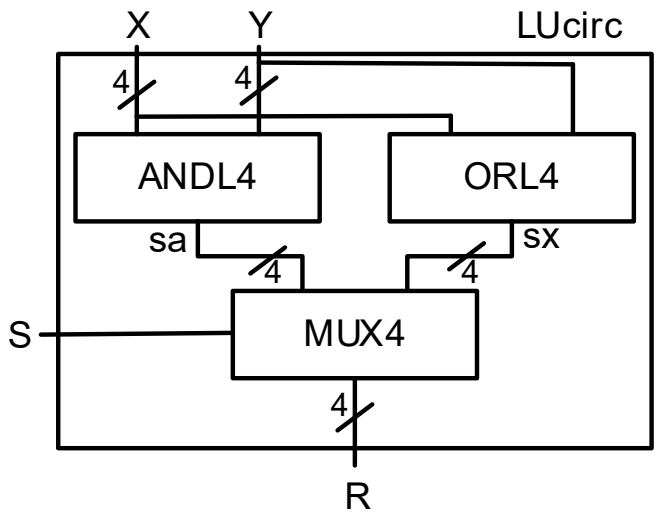
Descrição do Circuito Lucirc – Versão 1

```
entity LUcirc is
  port(X, Y : in STD_LOGIC_VECTOR(3 downto 0);
          S : in STD_LOGIC;
          R : out STD_LOGIC_VECTOR(3 downto 0)
  );
end LUcirc;

architecture arq_LUcirc of LUcirc is

component ANDL4
  port(A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          O : out STD_LOGIC_VECTOR(3 downto 0));
end component;

component ORL4
  port(A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          O : out STD_LOGIC_VECTOR(3 downto 0));
end component;

component MUX4
  port(A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          sel : in std_logic;
          O : out STD_LOGIC_VECTOR(3 downto 0));
end component;
```

Descrição do Circuito LUcirc – Versão 1 (cont.)

ISEL

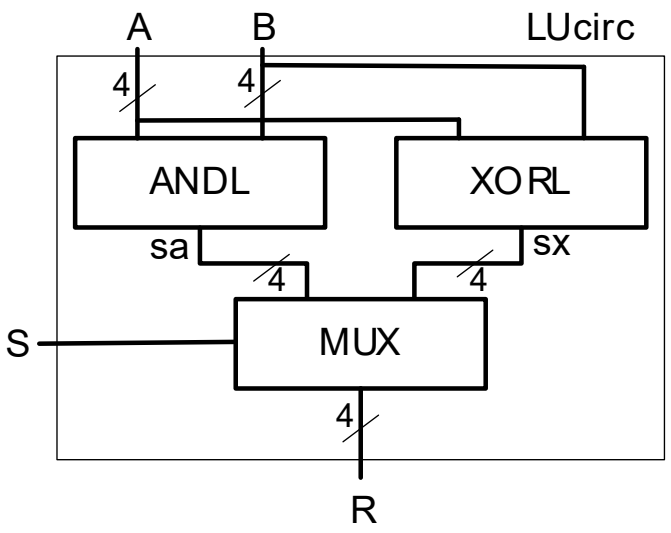
```
SIGNAL sa, sx : std_logic_vector(3 downto 0);

BEGIN

UANDL4: ANDL4 port map (
        A => X,
        B => Y,
        O => sa );

UORL4: ORL4 port map (
        A => X,
        B => Y,
        O => sx );

UMUX4: MUX4 port map (
        A => sa,
        B => sx,
        sel => S,
        O => R);

end arq_Lucirc;
```

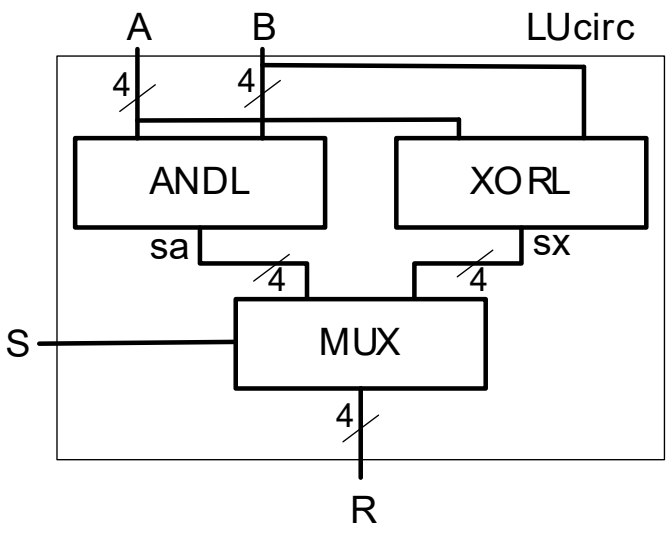


Descrição do Circuito LUcirc – Versão 2

ISEL

```
SIGNAL sa, sx : std_logic_vector(3 downto 0);

BEGIN

UANDL4: ANDL4 port map (
        A => X,
        B => Y,
        O => sa );

UORL4: ORL4 port map (
        A => X,
        B => Y,
        O => sx );

-- funções lógicas do multiplexer
R(0) <= (not S and sa(0)) or (S and sx(0));
R(1) <= (not S and sa(1)) or (S and sx(1));
R(2) <= (not S and sa(2)) or (S and sx(2));
R(3) <= (not S and sa(3)) or (S and sx(3));

end arq_Lucirc;
```



Descrição do Circuito LUcirc – Versão 3

```
SIGNAL sa, sx : std_logic_vector(3 downto 0);

BEGIN

sa(0) <= A(0) and B(0);
sa(1) <= A(1) and B(1);
sa(2) <= A(2) and B(2);
sa(3) <= A(3) and B(3);

sx(0) <= A(0) or B(0);
sx(1) <= A(1) or B(1);
sx(2) <= A(2) or B(2);
sx(3) <= A(3) or B(3);

-- funções lógicas do multiplexer
R(0) <= (not S and sa(0)) or (S and sx(0));
R(1) <= (not S and sa(1)) or (S and sx(1));
R(2) <= (not S and sa(2)) or (S and sx(2));
R(3) <= (not S and sa(3)) or (S and sx(3));

end arq_Lucirc;
```